VLSI System Design (Graduate Level)
Fall 2025

HOMEWORK I
REPORT

Must do self-checking before submission:

☑Compress all files described in the problem into one tar
☑ All SystemVerilog files can be compiled under SoC Lab environment
☑All port declarations comply with I/O port specifications
☑Organize files according to File Hierarchy Requirement
☑No any waveform files in deliverables

Student name: 楊淯絜

一、Summary

Hardware			
		RTL	synthesis
Top	CPU		
Synthesis result			
Area		Clock cycle(ns)	
$18832.299um^2$		$2.0\ ns$	
Firmware & Software			
	RTL pass	SYN pass	Execution time(ns)
Prog 0	✓	✓	74130
Prog 1	✓	✓	20805
Prog 2	✓	✓	73580
Prog 3	✓	✓	73090
Prog 4	✓	✓	71260
Prog 5	✓	✓	66690
Prog6	✓	✓	66390
Superlint(number of inline messages)			
Total lines	Warning	Error	coverage(%)
2822	132	0	95%

Table1. Summary of HW1

Write Your Text Summary Here:

- RISC-V CPU 實作

  設計一 5 stage pipelined CPU，並且在 ID Stage 進行分支預測，用兩個 32*32bits Register File 分別儲存整數類別及浮點數類別的資料，支援 49 條 RISC-V 指令。

- 測試和模擬

  藉 program0~program6 完整測試了各項指令運算及操作、排序演算法、浮點數計算、不用乘法指令完成乘法運算，分支預測功能也如預期正確運作。

- 指令集

  實現多種類型的指令，R-type、I-type、J-type、S-type、B-type、U-type 以及 M Extension(整數乘法)，還有浮點數運算相關的 F-type。

二、Simulation Report

RTL Simulation

```
DM[8245] = 40c10789, pass
DM[8246] = 404c5a18, pass
DM[8247] = 00000000, pass
DM[8248] = 000013fb, pass
your total cycle is 0.000000
your total cycle is 7396.000000

        ****************************
        **                        **       |__||
        **  Congratulations !!    **      / 0.0  |
        **                        **    /_____   |
        **  Simulation PASS!!     **   /^ ^ ^ \  |
        **                        **  |^ ^ ^ ^ |w|
        ****************************   \m___m__|_|

Simulation complete via $finish(1) at time 74130 NS + 2
../sim/top_tb.sv:94     $finish;
```

Program 0

```
DM[8221] = 73902c5d, pass
DM[8222] = 7a27c66f, pass
DM[8223] = 7b1ed6e1, pass

        ****************************
        **                        **       |__||
        **  Congratulations !!    **      / 0.0  |
        **                        **    /_____   |
        **  Simulation PASS!!     **   /^ ^ ^ \  |
        **                        **  |^ ^ ^ ^ |w|
        ****************************   \m___m__|_|

Simulation complete via $finish(1) at time 208050 NS + 2
../sim/top_tb.sv:94     $finish;
```

Program 1

```
DM[8192] = cfb90b0a, pass
DM[8193] = f56a8809, pass

        ****************************
        **                        **       |__||
        **  Congratulations !!    **      / 0.0  |
        **                        **    /_____   |
        **  Simulation PASS!!     **   /^ ^ ^ \  |
        **                        **  |^ ^ ^ ^ |w|
        ****************************   \m___m__|_|

Simulation complete via $finish(1) at time 73220 NS + 2
../sim/top_tb.sv:94     $finish;
```

Program 2

```
DM[8192] = 00000003, pass

        ****************************
        **                        **       |__||
        **  Congratulations !!    **      / 0.0  |
        **                        **    /_____   |
        **  Simulation PASS!!     **   /^ ^ ^ \  |
        **                        **  |^ ^ ^ ^ |w|
        ****************************   \m___m__|_|

Simulation complete via $finish(1) at time 73090 NS + 2
../sim/top_tb.sv:94     $finish;
```

Program 3

```
DM[8192] = 00375f00, pass
DM[8193] = 00000000, pass
DM[8194] = 0000129b, pass
your total cycle is 0.000000
your total cycle is 7110.000000

        ****************************
        **                        **       |__||
        **  Congratulations !!    **      / 0.0  |
        **                        **    /_____   |
        **  Simulation PASS!!     **   /^ ^ ^ \  |
        **                        **  |^ ^ ^ ^ |w|
        ****************************   \m___m__|_|

Simulation complete via $finish(1) at time 71260 NS + 2
../sim/top_tb.sv:94     $finish;
```

Program 4

```
DM[8192] = cfb90b0a, pass
DM[8193] = f56a8809, pass
DM[8194] = cfb90b0a, pass
DM[8195] = 0f95c456, pass
your total cycle is 0.000000
your total cycle is 0.000000

        ****************************
        **                        **       |__||
        **  Congratulations !!    **      / 0.0  |
        **                        **    /_____   |
        **  Simulation PASS!!     **   /^ ^ ^ \  |
        **                        **  |^ ^ ^ ^ |w|
        ****************************   \m___m__|_|

Simulation complete via $finish(1) at time 66680 NS + 2
../sim/top_tb.sv:94     $finish;
```

Program 5

```
DM[8192] = c11c1150, pass
DM[8193] = 433e05fc, pass
your total cycle is 0.000000
your total cycle is 0.000000

        ****************************
        **                        **       |__||
        **  Congratulations !!    **      / 0.0  |
        **                        **     /_____   |
        **  Simulation PASS!!     **    /^ ^ ^ \  |
        **                        **   |^ ^ ^ ^ |w|
        ****************************    \m___m__|_|

Simulation complete via $finish(1) at time 66390 NS + 2
../sim/top_tb.sv:94     $finish;
```

Program 6

Gate Level Simulation

```
DM[8246] = 404c5a18, pass
DM[8247] = 00000000, pass
DM[8248] = 000013fb, pass
your total cycle is 0.000000
your total cycle is 7396.000000

        ****************************
        **                        **       |__||
        **  Congratulations !!    **      / 0.0  |
        **                        **     /_____   |
        **  Simulation PASS!!     **    /^ ^ ^ \  |
        **                        **   |^ ^ ^ ^ |w|
        ****************************    \m___m__|_|

$finish called from file "/home/user1/avsd25/avsd25154/AVSD_2025_HW1_yu/./sim/top_tb.sv",
$finish at simulation time             74130000
```

Program0

```
DM[8220] = 66290483, pass
DM[8221] = 73902c5d, pass
DM[8222] = 7a27c66f, pass
DM[8223] = 7b1ed6e1, pass

        ****************************
        **                        **       |__||
        **  Congratulations !!    **      / 0.0  |
        **                        **     /_____   |
        **  Simulation PASS!!     **    /^ ^ ^ \  |
        **                        **   |^ ^ ^ ^ |w|
        ****************************    \m___m__|_|

$finish called from file "/home/user1/avsd25/avsd25154/AVSD_2025_HW1_yu/./sim/top_tb.sv", l
$finish at simulation time             208050000
```

Program1

```
Compiler version V-2023.12_Full64; Runtime version V-2023.12_Full64;  Oct  4 15:54 2025
Doing SDF annotation ...... Done

Done

DM[8192] = cfb90b0a, pass
DM[8193] = f56a8809, pass

        ****************************
        **                        **       |__||
        **  Congratulations !!    **      / 0.0  |
        **                        **     /_____   |
        **  Simulation PASS!!     **    /^ ^ ^ \  |
        **                        **   |^ ^ ^ ^ |w|
        ****************************    \m___m__|_|

$finish called from file "/home/user1/avsd25/avsd25154/AVSD_2025_HW1_yu/./sim/top_tb.sv", line 94.
$finish at simulation time             73580000
```

Program2

```
Compiler version V-2023.12_Full64; Runtime version V-2023.12_Full64;  Oct  4 15:55 2025
Doing SDF annotation ...... Done

Done

DM[8192] = 00000003, pass

        ****************************
        **                        **       |__||
        **  Congratulations !!    **      / 0.0  |
        **                        **     /_____   |
        **  Simulation PASS!!     **    /^ ^ ^ \  |
        **                        **   |^ ^ ^ ^ |w|
        ****************************    \m___m__|_|

$finish called from file "/home/user1/avsd25/avsd25154/AVSD_2025_HW1_yu/./sim/top_tb.sv", line 94.
$finish at simulation time             73090000
```

Program3

```
Compiler version V-2023.12_Full64; Runtime version V-2023.12_Full64;  Oct  4 15:56 2025
Doing SDF annotation ...... Done

Done

DM[8192] = 00375f00, pass
DM[8193] = 00000000, pass
DM[8194] = 0000129b, pass

        ****************************
        **                        **       |__||
        **  Congratulations !!    **      / 0.0  |
        **                        **     /_____   |
        **  Simulation PASS!!     **    /^ ^ ^ \  |
        **                        **   |^ ^ ^ ^ |w|
        ****************************    \m___m__|_|

$finish called from file "/home/user1/avsd25/avsd25154/AVSD_2025_HW1_yu/./sim/top_tb.sv", line 9
$finish at simulation time             71260000
           V C S   S i m u l a t i o n   R e p o r t
Time: 71260000 ps
```

Program4

```
Compiler version V-2023.12_Full64; Runtime version V-2023.12_Full64;  Oct  4 15:56 2025
Doing SDF annotation ...... Done

Done

DM[8192] = cfb90b0a, pass
DM[8193] = f56a8809, pass
DM[8194] = cfb90b0a, pass
DM[8195] = 0f95c456, pass

        ****************************
        **                        **       |__||
        **  Congratulations !!    **      / 0.0  |
        **                        **     /_____   |
        **  Simulation PASS!!     **    /^ ^ ^ \  |
        **                        **   |^ ^ ^ ^ |w|
        ****************************    \m___m__|_|

$finish called from file "/home/user1/avsd25/avsd25154/AVSD_2025_HW1_yu/./sim/top_tb.sv", line 94.
$finish at simulation time             66690000
```

Program5

```
Doing SDF annotation ...... Done

Done

DM[8192] = c11c1150, pass
DM[8193] = 433e05fc, pass

        ****************************
        **                        **       |__||
        **  Congratulations !!    **      / 0.0  |
        **                        **     /_____   |
        **  Simulation PASS!!     **    /^ ^ ^ \  |
        **                        **   |^ ^ ^ ^ |w|
        ****************************    \m___m__|_|

$finish called from file "/home/user1/avsd25/avsd25154/AVSD_2025_HW1_yu/./sim/top_tb.sv", line 94.
$finish at simulation time             66390000
           V C S   S i m u l a t i o n   R e p o r t
```

Program6

PA Report

Cycle Period	2ns
Cell Area :18832.299	

```
Library(s) Used:

    N16ADFP_StdCellss0p72vm40c (File: /usr/cad/CBDK/Executable_Package
    SRAM_ss0p72v0p72vm40c_100a (File: /home/user1/avsd25/avsd25154/M28

Number of ports:                          4994
Number of nets:                          24349
Number of cells:                         18133
Number of combinational cells:           15186
Number of sequential cells:               2898
Number of macros/black boxes:                2
Number of buf/inv:                        1498
Number of references:                       26

Combinational area:               6565.328800
Buf/Inv area:                      254.378888
Noncombinational area:            3184.220263
Macro/Black Box area:             9082.750000
Net Interconnect area:      undefined  (Wire load has zero net area)

Total cell area:                 18832.299063
Total area:                 undefined
1
```

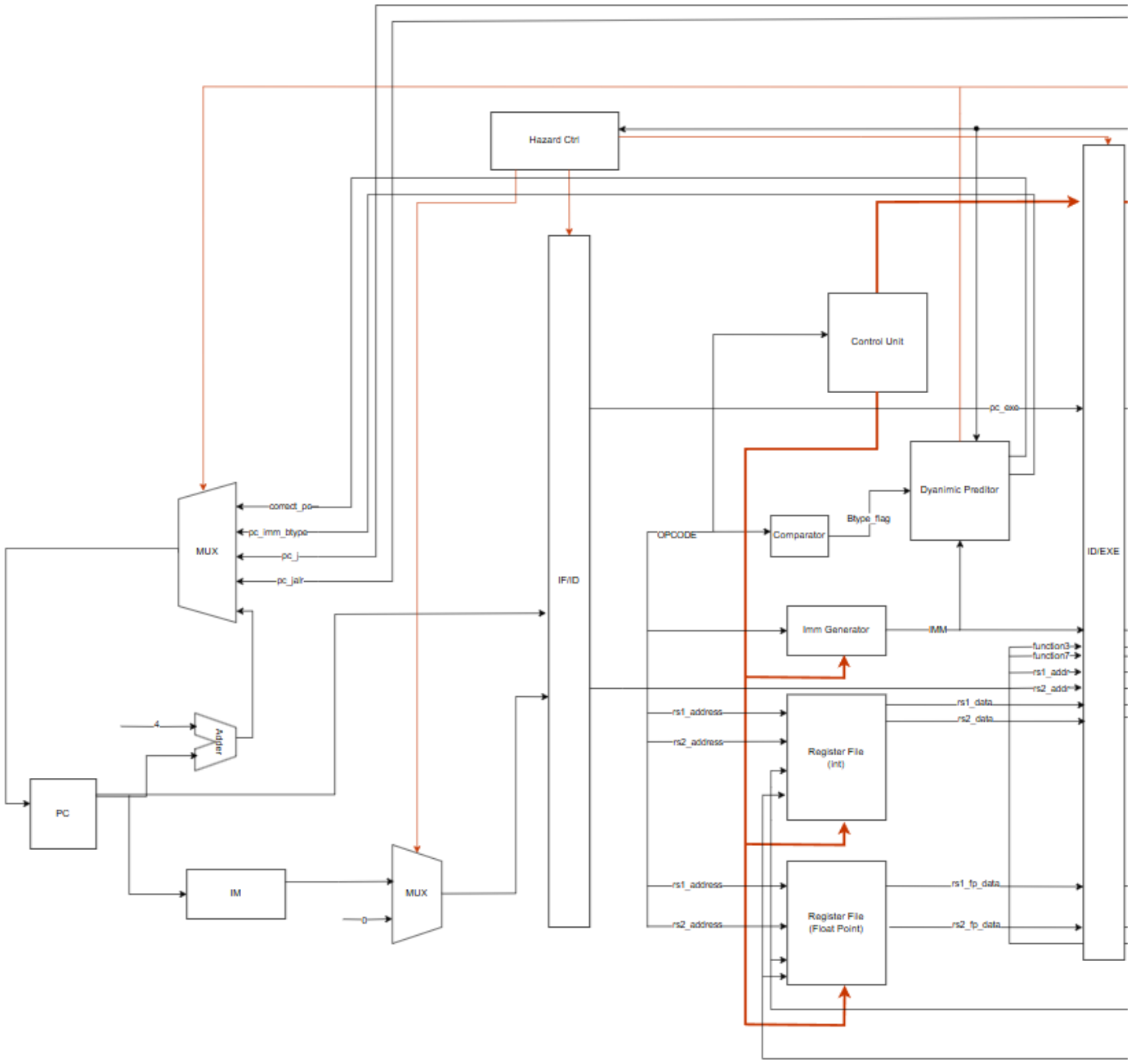
Superlint Coverage

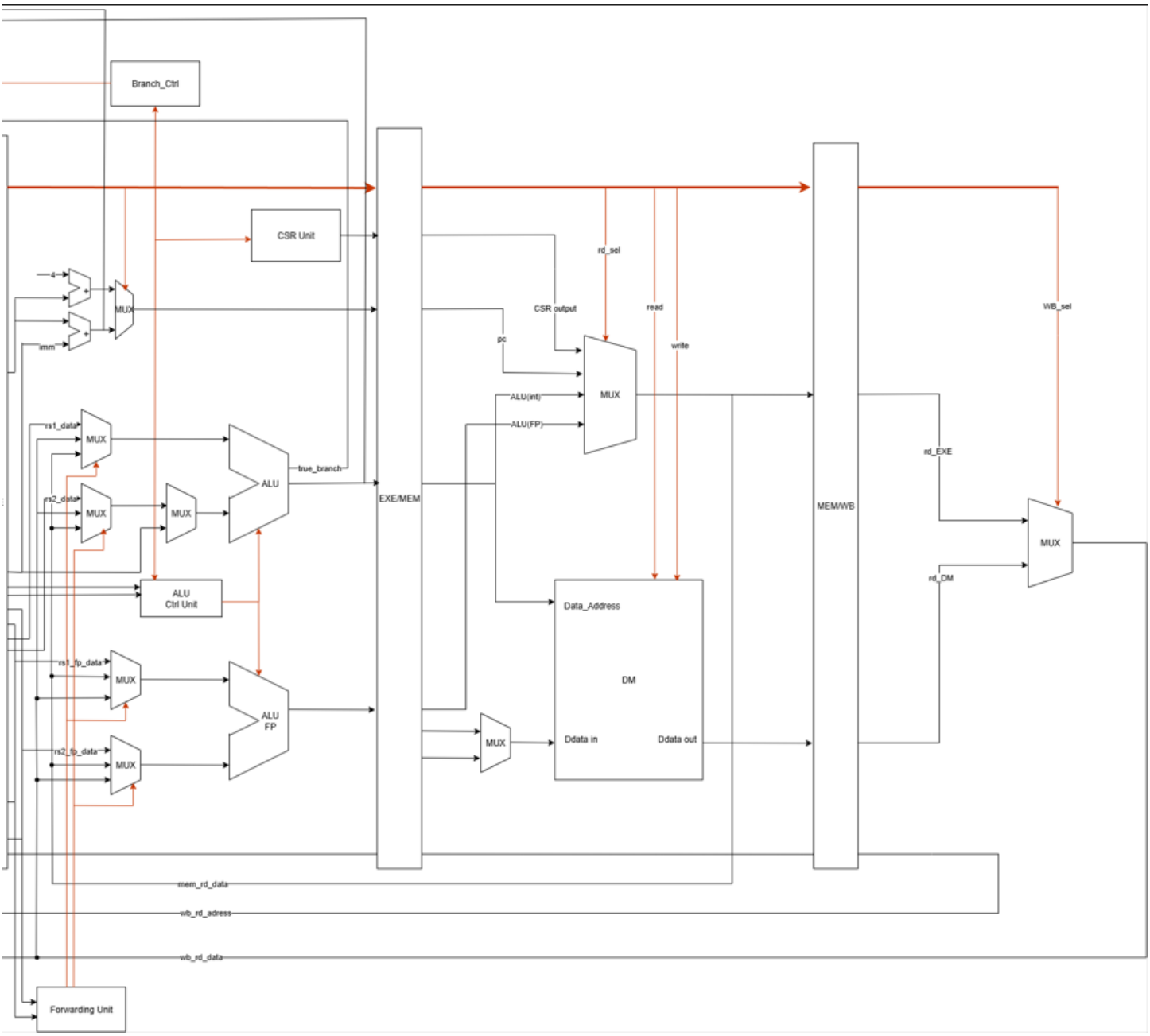
Violation Messages View

Description (Order by Category	
Category: CODINGSTYLE (80)	
Tag: CST_MS_LPDZ (7)	
Tag: ASG_MS_RPAD (2)	
Tag: WIR_NO_READ (5)	
Tag: CAS_NR_DEFA (3)	
Tag: CAS_NR_UCIT (2)	
Tag: ARY_NR_LOPR (5)	
Tag: ASG_NS_TRNB (43)	
Tag: OPR_NR_UEOP (7)	
Tag: OPR_NR_UREL (1)	
Tag: OPR_NR_UCMP (1)	
Tag: WIR_NO_USED (4)	
Category: STRUCTURAL (4)	
Tag: FLP_NO_ASRT (4)	

```
  37 ../src/Branch_Ctrl.sv
  69 ../src/CLZ.sv
  23 ../src/comparator.sv
 346 ../src/ControlUnit.sv
 115 ../src/CSR.sv
 305 ../src/Dynamic_Branch.sv
  97 ../src/EXE_ALU.sv
 131 ../src/EXE_ALU_Ctrl.sv
 144 ../src/EXE_FP_ALU.sv
 166 ../src/EXE_Stage.sv
  78 ../src/ForwardingUnit.sv
 101 ../src/Hazard_Ctrl.sv
  39 ../src/ID_FP_RegFile.sv
  35 ../src/ID_ImmGe.sv
  38 ../src/ID_RegFile.sv
 127 ../src/ID_Stage.sv
  22 ../src/IF_PC.sv
 134 ../src/IF_Stage.sv
 105 ../src/MEM_Stage.sv
  44 ../src/SRAM_wrapper.sv
   7 ../src/WB_Stage.sv
 659 ../src/top.sv
2822 total
```

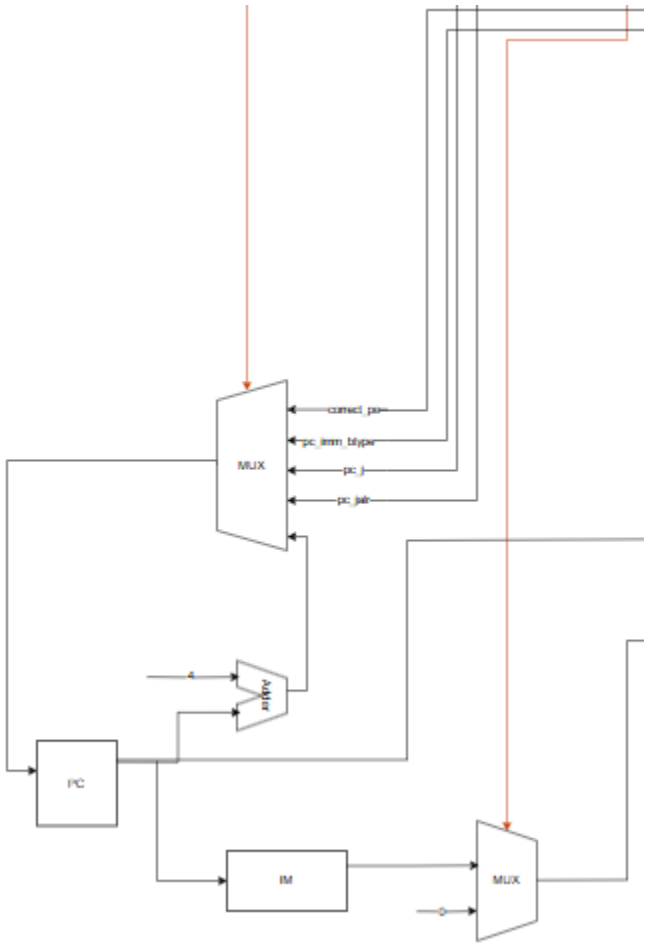
三、主要設計架構


Hazard Ctrl
Control Unit
pc_exe
Dyanimic Preditor
Btype_flag
Comparator
OPCODE
MUX
correct_pc
pc_imm_btype
pc_j
pc_jalr
IF/ID
ID/EXE
Imm Generator
IMM
function3
function7
rs1_addr
rs2_addr
rs1_address
rs2_address
rs1_data
rs2_data
Register File
(int)
Adder
4
PC
IM
MUX
0
rs1_address
rs2_address
Register File
(Float Point)
rs1_fp_data
rs2_fp_data


Branch_Ctrl
CSR Unit
4
imm
MUX
rs1_data
rs2_data
MUX
MUX
MUX
ALU
true_branch
ALU
Ctrl Unit
rs1_fp_data
rs2_fp_data
MUX
MUX
ALU
FP
EXE/MEM
rd_sel
CSR output
read
write
pc
ALU(int)
ALU(FP)
MUX
Data_Address
DM
Ddata in
Ddata out
MUX
MEM/WB
WB_sel
rd_EXE
rd_DM
MUX
mem_rd_data
wb_rd_adress
wb_rd_data
Forwarding Unit

四、子模組說明

1.IF Stage

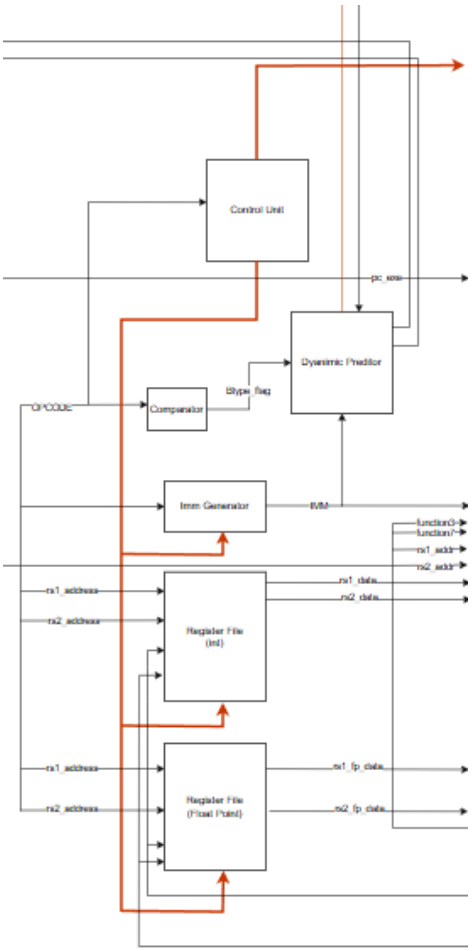


左圖為 IF Stage 架構圖，藉由一個 mux 去選擇正確的位址，圖中的 pc_imm_btype 為動態分支預測的跳轉位址，correct_pc 是如果在分支預測發生錯誤時進行修正用的正確位址，而 pc_j 和 pc_jalr 則分別是發生 jal 和 jalr 時會使用的位址。

- PC Module & Instruction Memory

在選到正確的位址後會經由 PC 向 IM 傳遞位址讀出對應的指令，後面的 mux 則是當需要進行 instruction flush 時會選擇 0，一般執行時皆是將指令向 ID Stage 傳遞

2.ID Stage



左圖為 ID Stage 的設計架構圖，在收到從 IF Stage 傳來的指令後會在 ID Stage 進行解碼，會在這邊將指令中的 opcode、rs1、rs2、rd、imm 分別提出。可以看到有以下子模組，Control Unit、Dynamic Preditor、Comparater、Imm Generator、Register file(int)、Register file(Float Point)，Dynamic Preditor、Comparater 模組會在下個小節進行說明。

- Control Unit

Control Unit 會在收到指令的 opcode 後輸出當條指令的類別以及相應的多條控制訊號

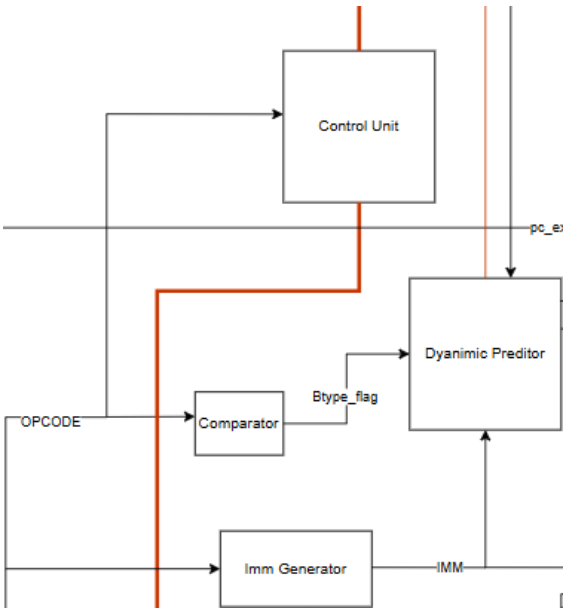
- Imm Generator

Imm Generator 會在接收到從 Control Unit 發來的控制訊號後傳出當條指令對應的正確的立即數。

- Register File(int) & Register File(Float Point)

此二 Register File 分別對應整數和浮點數運算所用，配合不同指令的暫存器讀寫操作

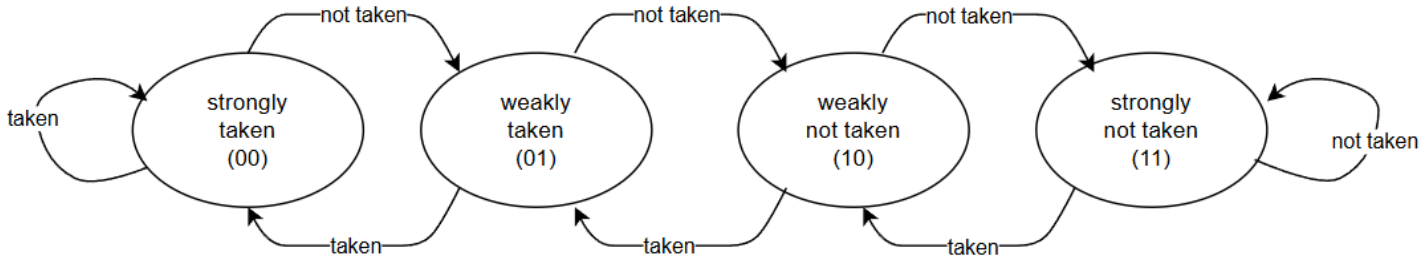
3.Dyamic Branch Preditor



左圖為 Comparator & Dyanimic Preditor 架構圖，指令 opcode 傳入 Comparator 後能得知當前指令是否為 B-type 指令，再將其判斷結果傳入 Dyanimic Preditor。

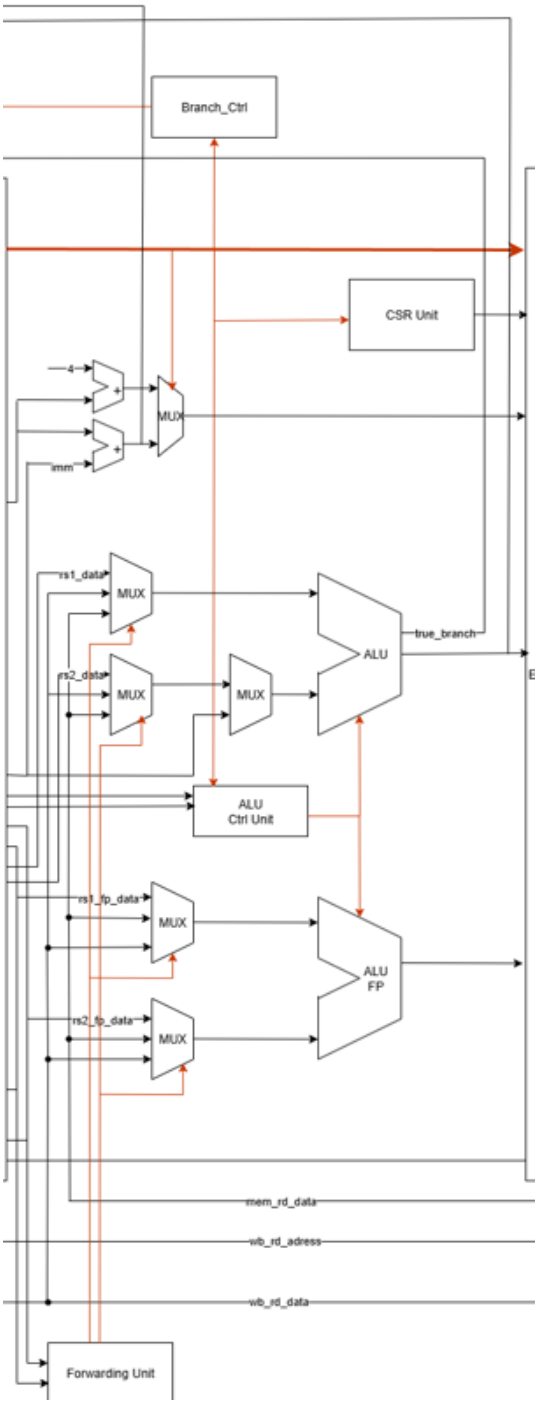
- Dyanimic Preditor

此模組目的為在 ID Stage 直接傳出跳轉位址回 IF Stage，以降低 B-type 指令所需使用的時脈數，達到提升效能之目標，其使用的狀態機行為以及編碼如下圖所示，共有四個狀態從 strongly taken 開始執行，即為在 ID Stage 預測當前 B-type 指令要進行跳轉，傳出跳轉位址至 IF Stage，若是預測錯誤則會至 weakly taken，一樣會在下次 B-type 指令出現時預測跳轉，若是又預測錯誤才會到 weakly not taken，這時便會預



測 B-type 指令沒有要跳轉，若是此時預測正確(確實沒有要跳轉)，便會到 stronly not taken，繼續預測不跳轉。預測跳轉也是同理，進行狀態更新。

4.EXE Stage

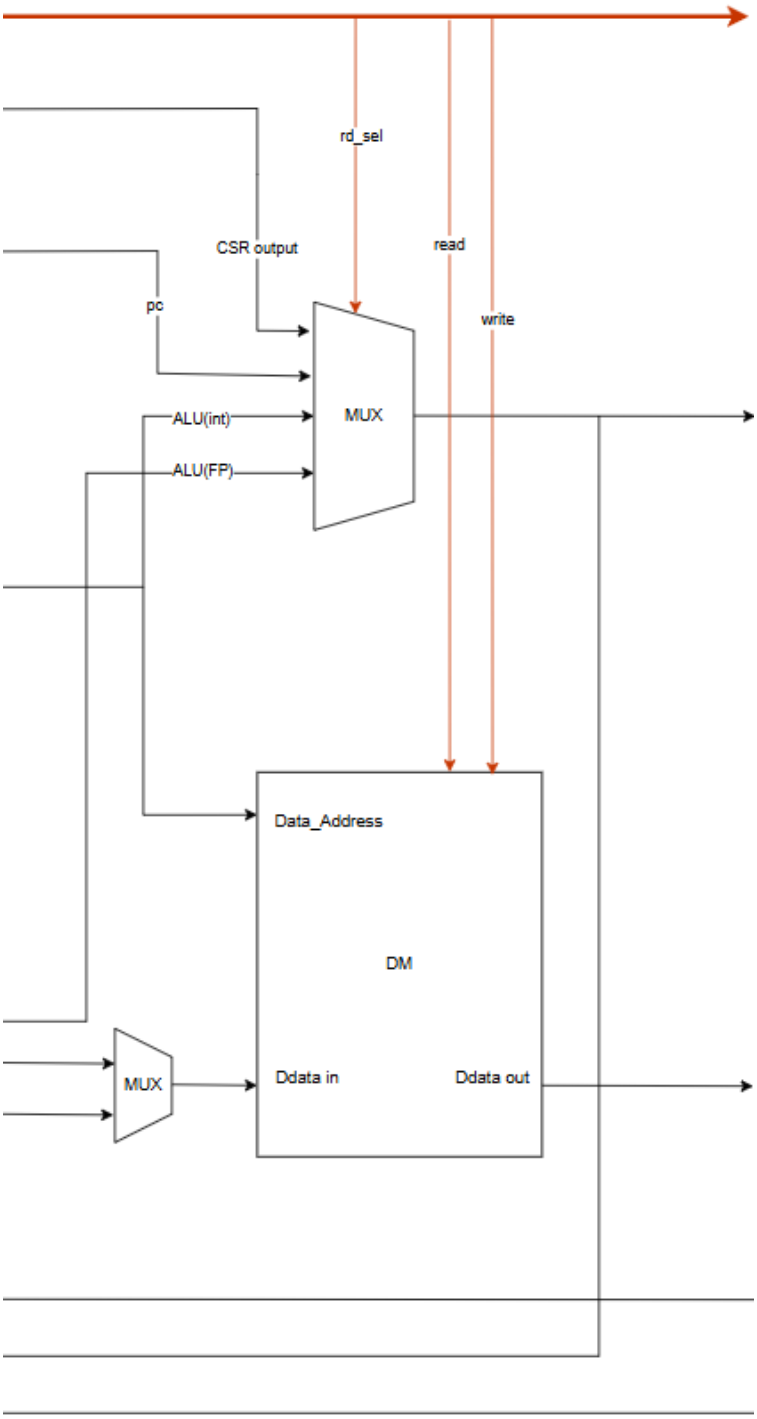


左圖為 EXE S tage 架構圖，分別有以下子模組 J&Jalr Ctrl Unit、CSR Unit、PC Mux、ALU(int)、ALU(FP)、ALU Ctrl Unit 、Forwarding Unit、Hazard Unit。

●J&Jalr Ctrl Unit
此模組為當 J 或 Jalr 指令傳遞至此時，傳出相應的位址選擇線至 IF Stage 進行控制，選取正確的位址。
●CSR Unit
CSR 模組在此作業中負責計算實際執行的指令數量以及所需要的週期數量，並在最後回傳
●PC Mux
此多工器目的是因應部分指令需回傳 pc+imm 或是 pc+4 作為 rd 的回傳，所以在此進行選擇。
●ALU(int)
此為針對整數運算做使用的核心運算模組。
●ALU(FP)
此為針對浮點數運算做使用的核心運算模組。
●ALU Ctrl Unit
此控制單元為接收從 Control Unit 傳遞至此的控制訊號對兩個 ALU 分別進行進一步的運算控制，
●Forwarding Unit
此模組會偵測當前運算指令是否有和前面的指令發生資料相依的情況，若是有的話會提前將 rd 資料直接傳回 EXE Stage 供當前指令進行運算使用，藉此可以避免指令選到錯誤的值造成後續計算的錯誤，
●Hazard Unit
此模組目的是在 CPU 執行指令時會遇到的 lw-use 以及指令跳轉狀況進行 pipeline 的暫停和 flush。當遇到 lw-use 時，由於正確的值還沒有回傳，所以需要先將指令

暫停，等待值從 WB Stage 回傳才繼續進行運算;而遇到需要跳轉指令位址時，由於在 CPU 知道是否需要跳轉前，下一條指令早已緊跟於當前指令了，所以確定發生跳轉時便需要由 Hazard Unit 視當前情況選擇要 flush 掉哪些指令(被跳過無須執行的指令)。

5.MEM Stage



左圖為 MEM S tage 的架構圖，由選擇 rd 的 mux 和 Data Memory 組成。

- RD MUX

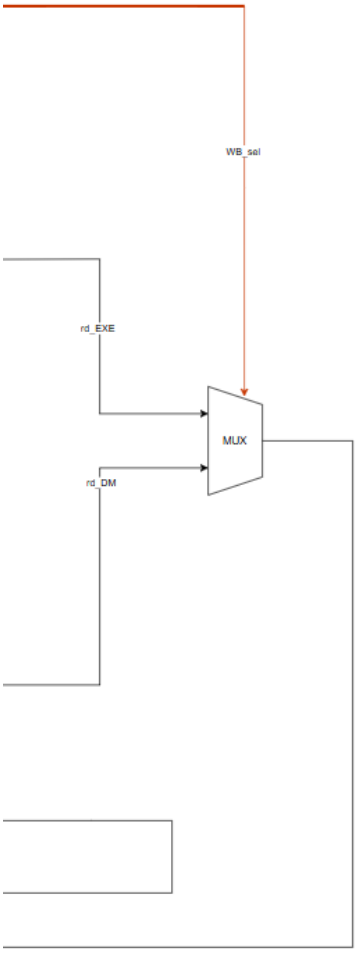
在 MEM Stage 會針對指令需要寫回 register file 的資料進行選擇，因應目前指令分別有 CSR 模組的輸出、pc(在 EXE Stage 進行初步選擇)、ALU(int)、ALU(FP)，

- Data Memory

load word 指令，會在 MEM Stage 進行 DM 操作，load word 會將在 EXE Stage 算出的 address 傳入 DM 並和 read 控制訊號配合將對應的資料讀出，傳至 WB Stage。

store word 指令，同樣會將在 EXE Stage 算出的 address 和從 register file 中讀出的資料傳至 MEM Stage 中，配合 write 訊號將資料寫入其 address 所指定的位置。

6.WB Stage



左圖為 WB Stage 的架構圖，其目的是將 EXE Stage 運算出的結果或是從 MEM Stage 讀出的資料回傳至 ID Stage 中的 register file。

- rd_EXE & rd_DM

在 WB Stage 中資料的選擇有 rd_EXE 和 rd_DM 兩種，rd_EXE 分別會因為在 MEM Stage 的控制訊號改變有以下四種，CSR_output(對應 CSR 指令)、pc(對應需要將和 pc 相關的資料寫回的指令)、ALU(int)、ALU(FP)；rd_DM 則是 load 指令從 Data Memory 中讀出的資料。

五、波形圖驗證

一、IF Stage

lead_branch_sel[3:0]	
4’b1000	correct_pc
4’b0100	Btype_pc_imm_IF
4’b0010	pc_mux_imm_rs1
4’b0001	pc_mux_imm

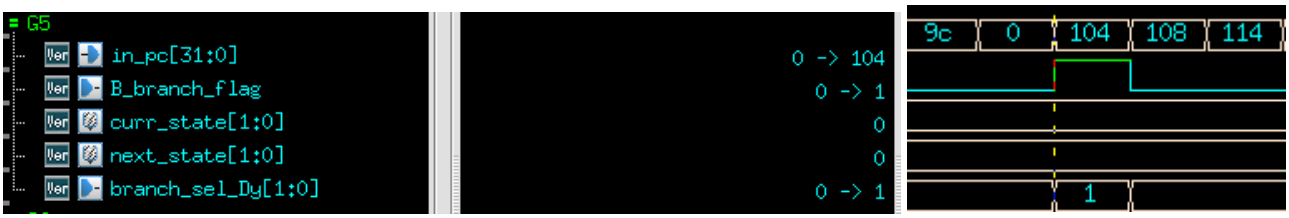
在 IF Stage 中共有五條不同的 pc 線視情況供選擇，分別是一般執行下一條指令的 PC_4、當分支預測錯誤時使用的 correct_pc、J-type 用的 pc_mux_imm、Jalr 用的 pc_mux_imm_rs1 以及 Btype 指令用的 Btype_pc_imm_IF，這五條訊號由 lead_branch_sel 做選擇，如上圖所示由於位址 0x104 的指令是 B-type，所以待 PC_in 傳出至 ID_Stage 後，進行分支預測，所以選中了 Btype_pc_imm_IF 得出的 0x114。

二、ID Stage

1.comparator

當指令從 IF Stage 傳入 ID Stage，OPCODE 會傳入 comparator 判斷當前此指令是否為 B-type 指令，如下圖所示，若是 B-type 指令就會拉起，B_branch_flage 至 Dynamic_Branch 模組進行後續處理。

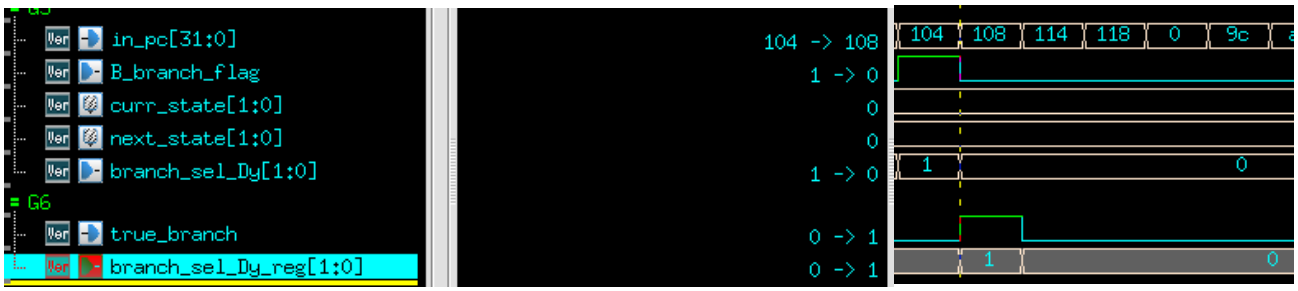
2.Dynamic_Branch



branch_sel_Dy	
2’b00	taken
2’b01	not taken
2’b10	correct_pc_sel

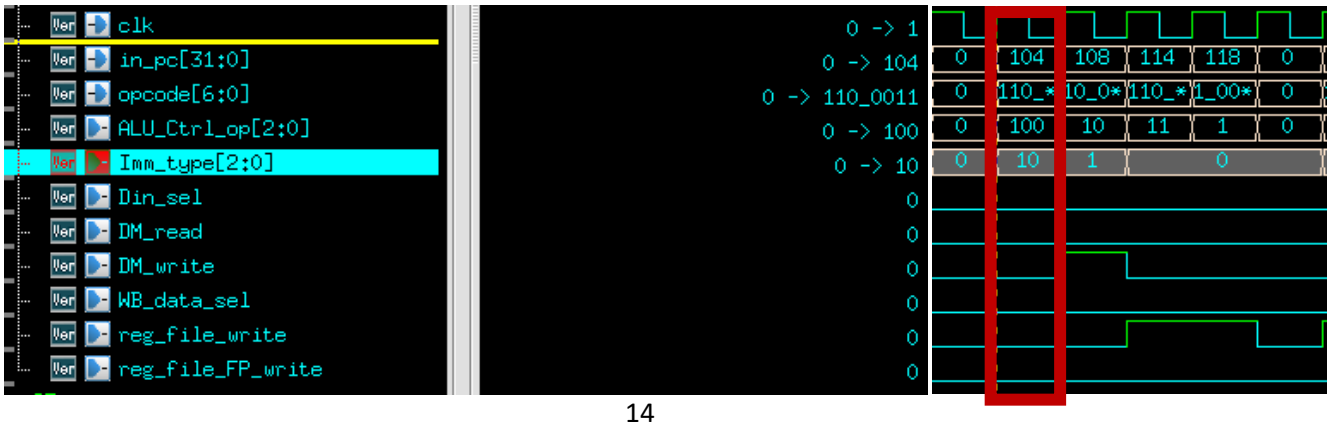
在接收到 B_branch_flag 訊號時，Dynamic_Branch 會根據當前狀態輸出猜測跳或是不跳的訊號，如上圖可以看到當前狀態(curr_state)是 0，預測要跳轉(如前述狀態機圖所述)，故 branch_sel_Dy 輸出 1 進行跳轉。

當指令走到 EXE Stage 時會確認這條指令的跳轉預測是否正確，藉由將 branch_sel_Dy 的輸出保留一個 clock(下圖的 branch_sel_Dy_reg)，和從 EXE Stage 傳回的 true_branch 訊號進行判斷是否要輸出 correct_pc_sel 進行 pc 跳轉的修正，如下圖所示，true_branch 和 branch_sel_Dy_reg 皆為 1，表示本次預測正確無須修正，branch_sel_Dy 輸出 0。



3.Control Unit

ControlUnit 在收到當前指令的 opcode 會輸出相應的控制訊號控制後續的運算，這邊同樣以位址 0x104 的 B-type 指令做說明，B-type 指令所需要輸出的相關控制訊號如下



表。ALU_Ctrl_op 是傳遞至 EXE Stage 中的 ALU_Ctrl 模組做使用，告知其指令型態為 B-type，讓它能夠對 ALU 單元做相應的控制；IMM_type 則是傳遞給同為在 ID Stage 的 IMM generator 模組，讓此模組知道該解碼出哪種型態的指令所需要的立即數，在這邊 3’d2 表示的就是 B-type 的立即數；Din_sel、DM_read、DM_write 這三條訊號是傳至 MEM Stage 進行 Data Memory 操作的控制訊號，由於 B-type 指令無須操作 DM 所以這邊都是 0；reg_file_write、reg_file_FP_write 這兩條則是控制兩個 register file 的寫入操作，但因為 B-type 指令沒有需要寫回的資料所以這邊也同樣都是 0。

ALU_Ctrl_op	3’b100
IMM_type	3’d2
Din_sel	1’b0
DM_read	1’b0
DM_write	1’b0
wb_sel	1’b0
reg_file_write	1’b0
reg_file_FP_write	1’b0

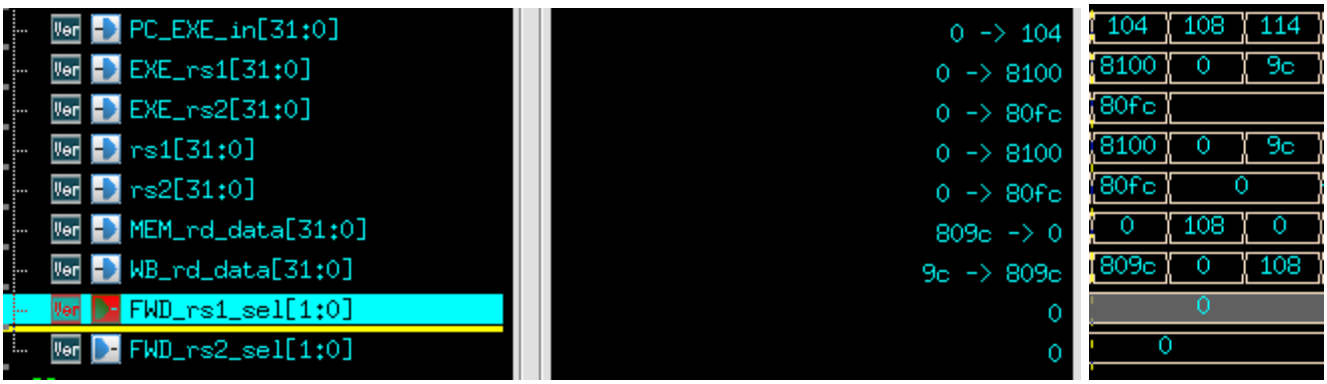
三、EXE Stage

- ForwardingUnit

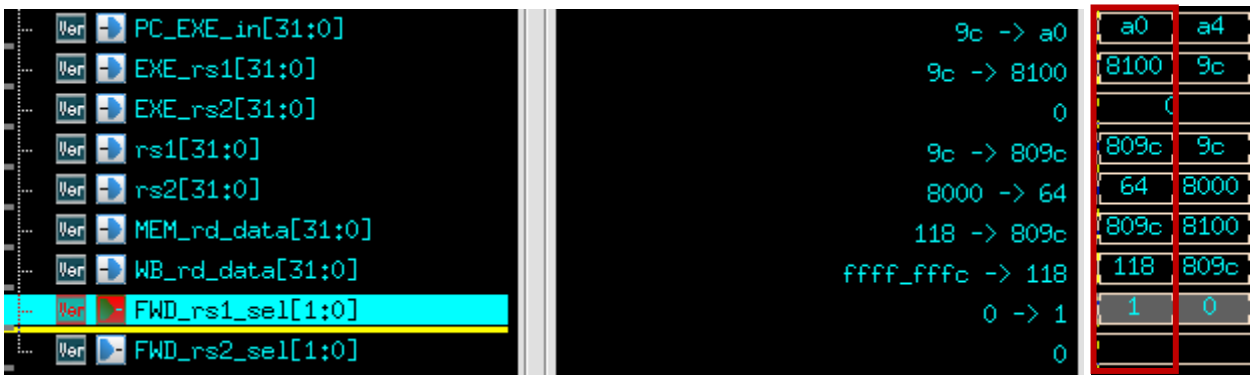
當訊號傳遞至 EXE Stage 時會由 ALU Ctrl 單元承接 Control Unit 的控制訊號去對 ALU 進行控制，而 rs1 和 rs2 在進入 ALU 進行運算前會先經過 MUX 去選擇，在此設計中針對資料相依的情況設定了兩條前饋，分別是從 MEM Stage 至 EXE Stage 以及 WB Stage 至 EXE Stage，MUX 的控制則由 Forwarding Unit 負責，如下表所示。

從下方波形圖可以看到當前 0x104 指令為 B-type，當其在 EXE Stage 要執行運算時並沒有跟前面指令發生資料相依的問題，所以 rs1、rs2 的 FWD_sel 皆選 0，直接使用從 ID_Stage 傳入的 rs1、rs2 做運算。

Forwarding_sel	
0	選擇從 ID Stage 傳入的 rs1、rs2
1	選擇從 MEM Stage 傳回的 rs1、rs2
2	選擇從 WB Stage 傳回的 rs1、rs2



下方波形圖表示的是一遇到資料相依情況的變化，由於 0x9c 指令為 auipc a0,0x8，而 0xa0 則是 addi a0,a0,100，在此發生了資料相依問題，因此可以看到波形圖中的 FWD_rs1_sel 跳轉至 1，選取了從 MEM Stage 傳回的值進行運算。



- ALU

待正確的 rs1、rs2 被選取後便會進去 ALU 進行運算，下列將針對不同種類的指令運算做舉例說明。

➢ R-type

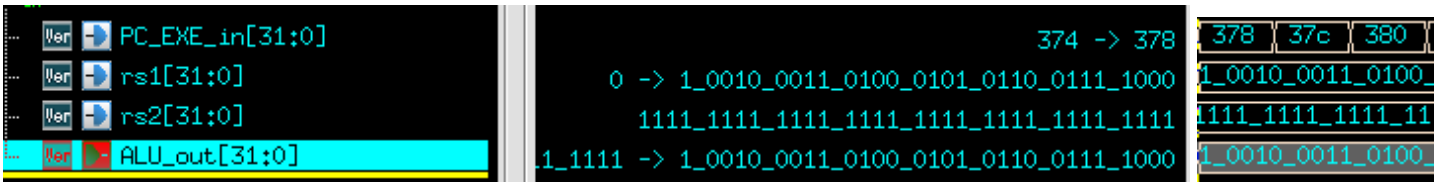
1.ADD

如下圖所示當執行到 0x130 時，指令為 add t0,t0,t1，t0、t1 分別對應 rs1、rs2 其值皆為-1，執行 ADD 運算後輸出為-2。

2.AND

如下圖所示，當執行到 0x378 時，指令為 and t0, t0, t1，其運算如表格所示(由於是 and 運算表格比較好看)

rs1	0001_0010	0011_0100	0101_0110	0111_1000
rs2	1111_1111	1111_1111	1111_1111	1111_1111
ALU_out	0001_0010	0011_0100	0101_0110	0111_1000



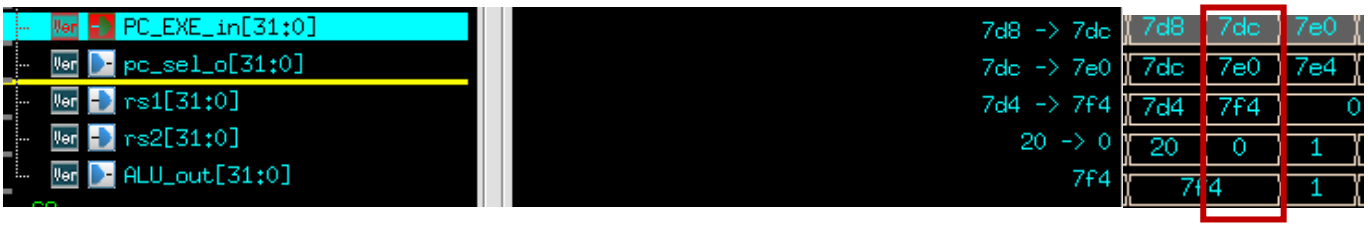
3.MUL

如下圖所示，當執行到 0x3bc 時，其指令為 mul t0, t0, t1，將波形轉為十進制方便確定運算結果，rs1 為 4660，rs2 為 15，相乘得出 69900 正確無誤。

➢ I-type

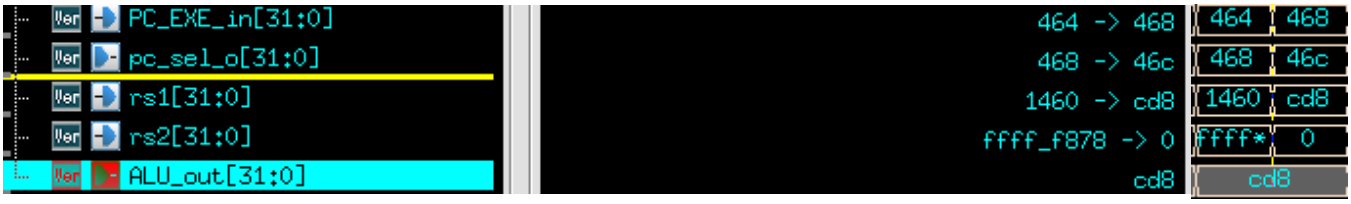
1.JALR

如下圖所示，當執行到 0x7dc 時，指令為 jalr t1,t1，jalr 的 PC=imm+rs1，所以可以看到 imm 為 0 被選入 rs2 和 rs1 算出跳轉位址 7f4，而 jalr 還需要回傳當前的 pc+4 作為 rd，可以看件 pc_sel_o(在 EXE Stage 選擇傳入 WB Stage 的訊號)的值為 7e0，正確無誤。



2.LW

如下圖所示，當執行到 0x468 時，指令為 lw t0,0(t1)，此指令的 imm+rs1 的值為 0xcd8，根據下圖可以看到已經正確計算出位址，接下來 LW 指令會從 Data Memory 中讀

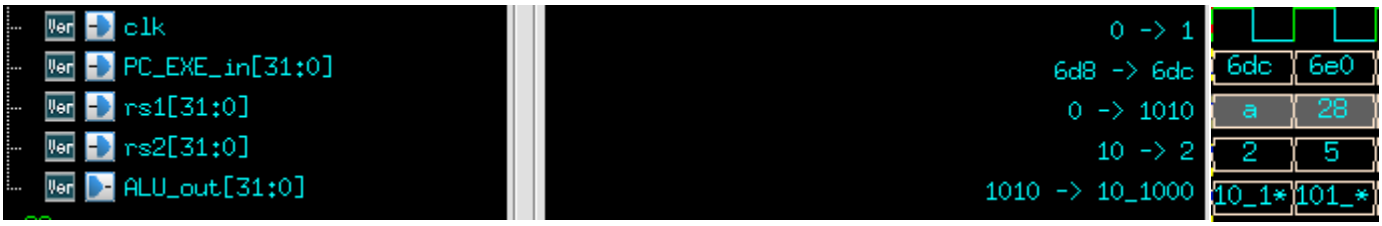


出對應位址的 Data 進行回傳，可以從下圖看到 DO 為從 Data Memory 讀出資料的端口，於 clock 負緣讀出並在下個 clock 正緣傳至 WB Stage。

3.SLLI

如下圖所示，當執行到 0x6dc，指令為 SLLI，其目標為對 rs1 進行邏輯左移，可以看到 shamt 為 rs2=0x2，rs1 是 0xa(二進制為 1010)

，經過運算後可以看到輸出確實是 1010 左移二位，運算正確。



➢ S-type

1.SW

如下圖所示，當執行到 0x874 時，指令為 sw t0,-4(s0)，sw 指令，其位址計算為 rs1+imm，可以看到立即數作為 rs2 被選入進行運算，後面將其結果取[15:2]傳入 Data Memory 並存入資料。

➢ B-type

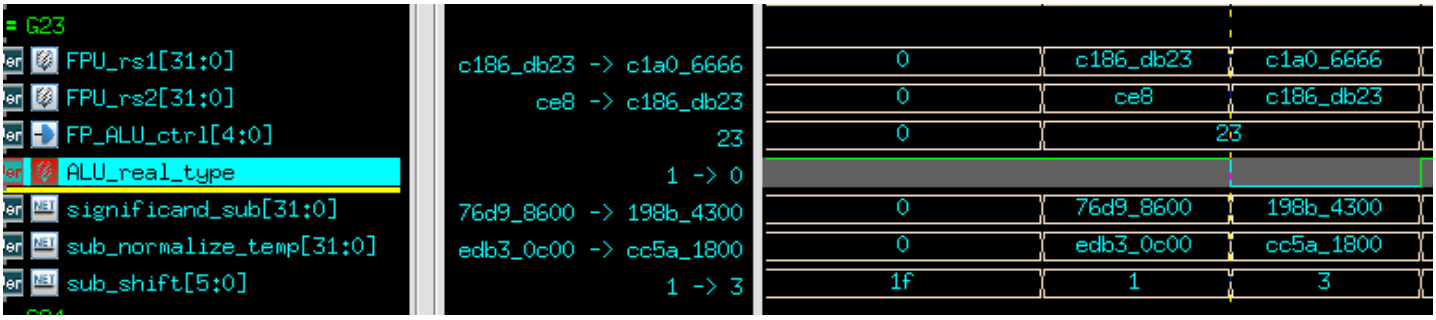
1.beq

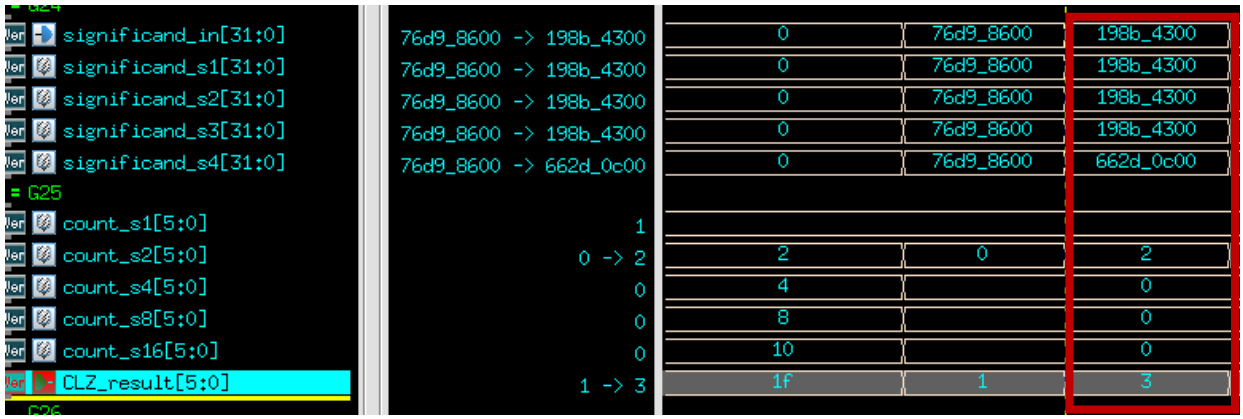
如下圖所示，當執行到 0x8cc 時，指令為 beq t0,t1,904，rs1 若和 rs2 相等則跳轉，rs1 為 ffff_f000，rs2 為 ff0，兩者不相等因此不跳轉，ALU_out 會做為前述在 Dynamic Branch Preditor 中提到的 true_branch 訊號傳回 ID_Stage 做狀態更新，同時也做為是否需要修正的依據。

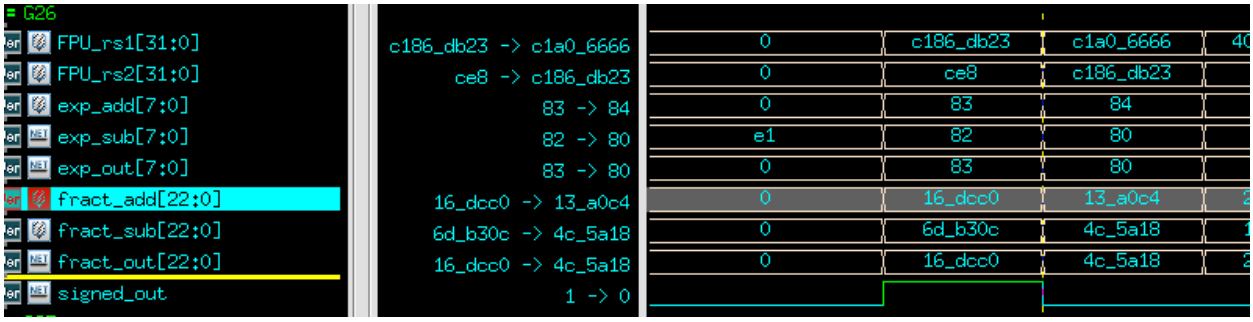
➢ F-type

1. FADD.S 和 FSUB.S

浮點數計算採用 IEEE-754 標準，將其分為 signed bit、exp、fraction 三個部分，且因為減法可能會出現比原本 exp 還小的情況，所以加入 CLZ 進行 fraction 的修正。下圖 CLZ_result 即為實際要移的 bits 數，最後再將 signed bit、fraction、exp 重組，得到結果







➢ J-type

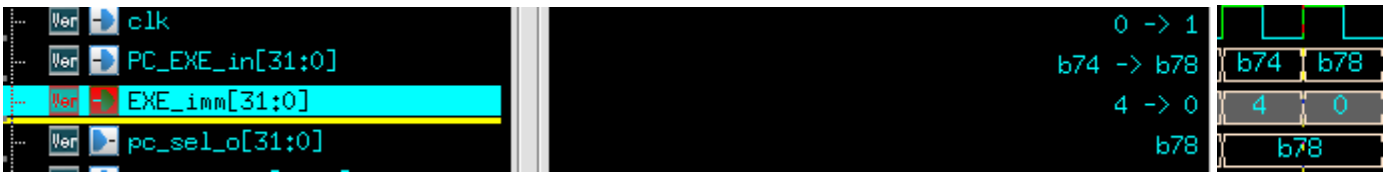
1.JAL

如下圖所示，當執行到 0xbb4 時為 jal 指令，jal 指令的跳轉 pc 為當前 pc+imm(即為圖中的 pc_mux_imm)，在此設計中是將其做在 ALU 外面，而 rd 則須要回傳當前 pc+4，可以觀察到在 EXE Stage 執行完 jal 後第二個 clock，也就是 WB Stage 順利選到 0xbb8 回傳。

➢ U-type

1.AUIPC

如下圖所示，當執行到 0xb78 時，指令為 auipc t0,0x0，其目的為將 pc+imm 做為 rd 回傳，可以看到當下 imm 為 0 和 0xb78 相加後仍然是 0xb78，並透過 pc_sel_o 向 MEM Stage 傳遞，和 jal 雖然同樣是 pc+imm 但不同是 jal 是要做為下一個 pc 要跳轉的位址，要立即回傳至 IF Stage 中，而 auipc 則是要將其回傳至 register file 中。



- Hazard

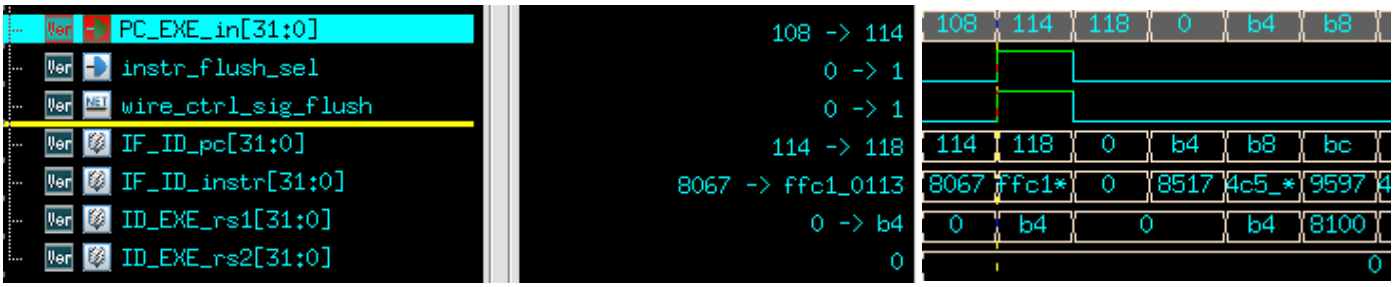
此模組目標是在出現指令跳轉或是 lw-use 時，需要讓後面指令失效或是暫停，以下藉 B-type 以及 JALR 指令以及 lw-use 做說明。

1. B-type

如下圖所示此 pc(於 ID Stage)，執行至 0x104 時，指令為 bgeu a0,a1,114，由於此次新增分支預測於 ID Stage，所以可以看到 B_branch_flag 訊號拉起，且當前狀態為 0，表示猜測要跳，此時將跳躍位址 0x114 傳入 program counter，下個 clock 可以看到 true_branch 訊號拉起，表示此次確實要跳躍，預測正確，Hazard 模組偵測當前狀況，判斷出要將已經在 ID Stage 的 0x108 指令清除，所以 wire_ctrl_sig_flush 拉起，可以看到 rs1、rs2 隨後被清為 0。至此完成了分支預測目標。

2.JALR

如下圖所示，pc 於 EXE Stage 執行到 114 時，指令為 ret，其行為屬於 JALR，JALR 和 J-type 指令皆在 EXE Stage 進行判斷已經位址跳轉選擇，所以此時我們需要清除已經在 IF Stage、ID Stage 的指令，才能順利完成跳轉，因此如下圖所示，當 Hazard 得知當前為 JALR 指令，將 instr_flush_sel、wire_ctrl_sig_flush 拉起，分別將 IF Stage 準備匯入 pipeline(IF/ID)的指令(0x11c)清 0，以及準備從 ID_Stage 匯入 pipeline(ID/EXE)的指令(0x118)清除，可以看到前述的值皆順利清為 0，完成 JALR 跳轉，J-type 同理。



3.lw-use

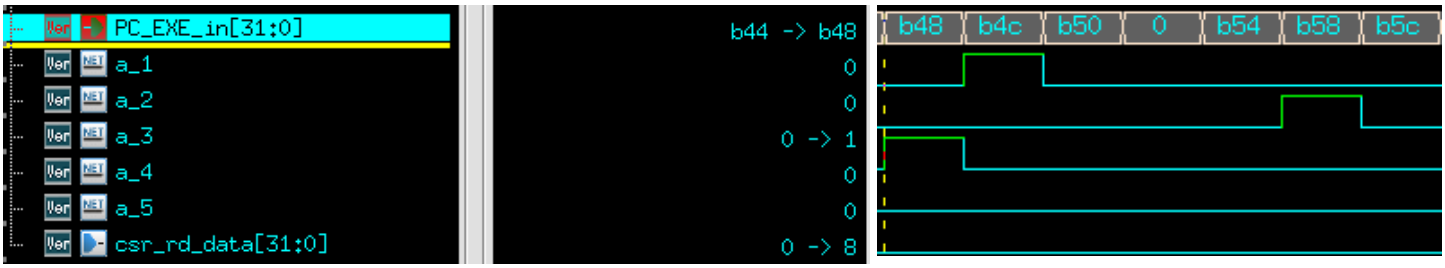
如下圖所示，pc 於 EXE Stage 執行到 0x214、0x218，指令分別為 lw a5,-20,(s0)、addi a5,a5,1，由於 0x218 會立刻使用到 a5，所以這邊遇到了 lw-use，此刻 lw_use 訊號拉起，令 IF_ID_reg_write 及 pc_write 降為 0 停止 IF/ID Stage 之間 pipeline 的寫入和 program counter 的輸出，藉此將 0x218 指令暫停，不被後面指令覆寫，wire_ctrl_sig_flush 拉起，將尚為得到正確 a5 的 0x218 指令轉為 nop，避免錯誤的計算產生。

- CSR

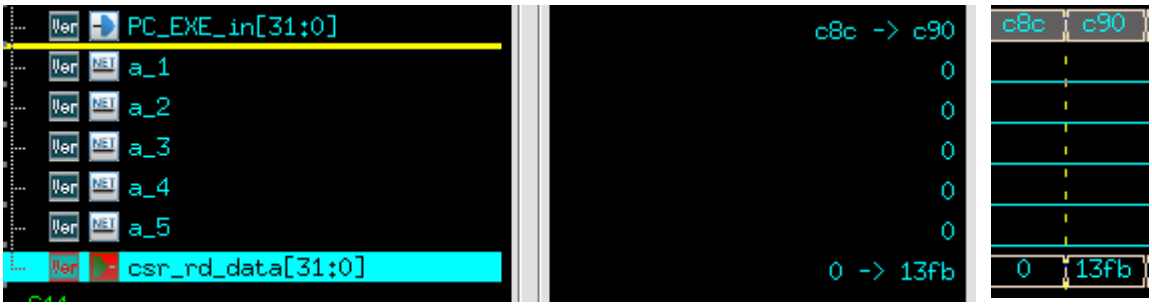
此模組於本次作業目的為計算執行指令總數以及所花費的 cycle 數量，由於此次加入分支預測，所以會有多種不同的清除指令狀況，影響指令數量計算，分類如下表。

預測和實際狀況	對應須清除的指令數量
a_1=>J-TYPE & JALR-TYPE	2
a_2=>guess not taken but false	2
a_3=>guess not taken and true	0
a_4=>guess taken but false	2
a_5=>guess taken and true	1

如下圖所示，0xb48 為 bgeu t0,t1,b50、0xb4c 為 j b54、0xb58 為 bgeu t0,t1,b60，根據各自跳轉情況 a_1~a_5 訊號對應拉起，匯入 CSR 模組，計算實際執行的指令數量。



最後輸出本次測試執行指令總數，以 prog 0 為例，如下圖所示。



四、MEM Stage

在這邊除了將要寫回 Register File 的資料回傳外，最重要的便是操作 Data Memory 的讀寫。

1.Store

如前述架構圖，要寫入 DM 的值會是 rs2 的資料，先經由一 MUX 選擇是整數還是浮點數，而寫入又有 byte、half、full write，藉由下圖 BWEB 訊號控制，為 active low，下圖舉例的 store 指令為 full write，因此為 0，可以看到 rs2 的資料被完整寫入 Data Memory(DI)。

2.Load

Load 指令目的是將 Data Memory 的資料讀出，當前執行到 0x4c0 為 lw 指令，將 0x90e8 從 Data Memory(DO)讀出，並由 data_sel 訊號選中後由 WB Stage 回傳。

五、WB Stage

此由一個二對一 mux 組成，負責選擇要回傳的資料，有以下兩種

1.當 data_sel 為 0 時，選擇從 MEM Stage 的 MUX 選出經由 EXE Stage 計算出的值

2.當 data_sel 為 1 時，選擇從 Data Memory 讀出的值，如同前述 Load 指令所示。

六、Lessons Learned

本次作業讓我將過往學習到的計算機組織知識藉由實作而理解的更加透徹，令我印象比較深刻的有以下幾點:

- Forwarding
  之前只在計算機組織課程中學過，經過這次實作才知道原來會使用到此功能的頻率是非常高的，而且有非常多的 Forwarding 選擇，本次是做了從 MEM Stage 至 EXE Stage 以及 WB Stage 至 EXE Stage 兩種。
- Hazard
  和 Forwarding 一樣在經過實作後，對各種指令的清除行為更加清楚，尤其是在加入分支預測後，有更多不同的指令清除情況需要做分類，還有在遇到 lw-use 時需要加入暫停 program counter 的設計。
- Branch Predictor
  在分支預測的設計上令我最有印象的 bug 是出在我沒有針對 lw-use 情況設計停止預測狀態的跳轉導致，整個狀態亂掉，剛開始我完全疏忽這點，導致後面花了不少時間往前回推才發現。